

Theoretical analysis of time complexity

Merge sort part

- Hybrid merge sort partitions the array in half until the size of each subarray $\leq S$
- Number of subarrays $\approx \frac{n}{S}$ hence number of levels $\approx \log_2(\frac{n}{S})$
- At each level, merging costs $O(n)$, hence merge sort part costs $O(n \log \frac{n}{S})$

Insertion sort part

- Total of $\lceil \frac{n}{S} \rceil$ insertion sort calls, each with at most S elements
- Each insertion sort costs $O(S^2)$
- Hence, total across all subarrays is $\frac{n}{S} \times O(S^2) = O(nS)$

Hybrid algorithm time complexity

- $T(n, s) = O(n \log \frac{n}{S}) + O(nS) = O(n \log \frac{n}{S} + nS)$

c(i): Fix S and change size n

Since $S = 10$ is fixed, theoretical time complexity = $O(n \log n + n) = O(n \log n)$

n	Comparisons	$\frac{\text{comparisons}}{n \log_2 n}$
1000	9092	0.912
10000	127054	0.956
100000	1557587	0.938
1000000	19071486	0.957
10000000	226426243	0.974

Since the ratio is roughly constant, the empirical results agree with this analysis

c(ii): Fix n and change threshold S

Since $n = 1000000$ is fixed, average-case model = $O(\log \frac{n}{S} + \frac{S}{4})$

- For random data, insertion sort on S elements makes $\frac{S^2}{4}$ comparisons on average
- Hence, time complexity of insertion sort part = $\frac{n}{S} \times \frac{S^2}{4} = \frac{nS}{4}$
- We use average-case model for actual number of comparisons

S	Comparisons	$\frac{\text{comparisons}}{\log_2 \frac{n}{S} + \frac{S}{4}}$
1	18673888	925294
2	18673888	961008
4	18728179	989257
8	19074428	1007546
16	20225265	1014735
32	23193587	1011426
64	29905361	999124
128	44234781	984492

Since the ratio is roughly constant, the empirical results agree with this analysis

c(iii): Optimal value for S

We find the optimal value for S experimentally using CPU time, not just number of comparisons

- As S increases, number of recursive calls (merge sort levels) decreases
- However, number of insertion sort comparisons increases

S	n	Time taken (s)
1	1000	0.0
1	10000	0.015625
1	100000	0.34375
1	1000000	4.34375
1	10000000	52.9375

S	n	Time taken (s)
2	1000	0.0
2	10000	0.03125
2	100000	0.265625
2	1000000	3.78125
2	10000000	50.765625

S	n	Time taken (s)
4	1000	0.0
4	10000	0.015625
4	100000	0.25
4	1000000	3.34375
4	10000000	51.875

S	n	Time taken (s)
8	1000	0.0
8	10000	0.03125
8	100000	0.21875
8	1000000	3.578125
8	10000000	48.234375

S	n	Time taken (s)
16	1000	0.0
16	10000	0.03125
16	100000	0.234375
16	1000000	3.265625
16	10000000	48.875

S	n	Time taken (s)
32	1000	0.0
32	10000	0.0
32	100000	0.234375
32	1000000	3.328125
32	10000000	50.453125

S	n	Time taken (s)
64	1000	0.0
64	10000	0.03125
64	100000	0.28125
64	1000000	4.265625
64	10000000	51.953125

S	n	Time taken (s)
128	1000	0.015625
128	10000	0.03125
128	100000	0.375
128	1000000	5.296875
128	10000000	59.984375

S	Average time taken (s)
1	11.5281
2	10.9688
4	11.0969
8	10.4125
16	10.4813
32	10.8031
64	11.3063
128	13.1406

Since it has the lowest average time taken, the optimal value for S is 8

- Largest dataset with n=10000000 dominates average time taken
- However, this is fine since we consider the performance of different S on huge datasets

d) Original vs hybrid merge sort

	Comparisons	CPU time (s)	n	S
Original merge sort	220099725	47.890625	10000000	8
Hybrid merge sort	221108960	44.250000	10000000	8

Hybrid merge sort performs 0.46% more comparisons than original merge sort

- However, it takes 7.60% less CPU time than original merge sort

Despite doing more comparisons, hybrid merge sort is faster

- Insertion sort part reduces overhead of recursive calls and merging for small subarrays